Design and Analysis of Algorithms
Application of Greedy Algorithms (II)

1. Huffman Coding
2. Single Source Shortest Path Problem
   - Dijkstra's Algorithm
3. Minimum Spanning Tree
   - The Cut-Based MST Algorithm
   - Kruskal's Algorithm
   - Prim's Algorithm

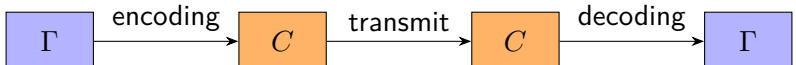
Outline

1. Huffman Coding
2. Single Source Shortest Path Problem
   - Dijkstra's Algorithm
3. Minimum Spanning Tree
   - The Cut-Based MST Algorithm
   - Kruskal's Algorithm
   - Prim's Algorithm

Motivation of Coding

Let Γ be an alphabet of n symbols, and let f_i be the frequency of symbol i. Clearly, $\sum_{i=1}^{n} f_i = 1$.

In practice, to facilitate transmission in the digital world (improve efficiency and robustness), we encode symbols (characters in a file) as codewords and transmit the resulting code.



What is a good coding scheme?

- efficiency: efficient encoding/decoding & low bandwidth
- other nice properties about robustness such as error-correcting

Fixed-Length vs. Variable-Length Coding

Two approaches to encoding a source (Γ, F)

- Fixed-length coding: the length of each codeword is a fixed number
- Variable-length coding: the length of each codeword could be different

Fixed-Length vs. Variable-Length Coding

Two approaches to encoding a source (Γ, F)

- Fixed-length coding: the length of each codeword is a fixed number
- Variable-length coding: the length of each codeword could be different

Fixed-length coding seems neat. Why bother to introduce variable-length coding?

Fixed-Length vs. Variable-Length Coding

Two approaches to encoding a source (Γ, F)

- Fixed-length coding: the length of each codeword is a fixed number
- Variable-length coding: the length of each codeword could be different

Fixed-length coding seems neat. Why bother to introduce variable-length coding?

- If each symbol appears with same frequency, then fix-length coding is good.
- If (Γ, F) is non-uniform, we can use less bits to represent more frequent symbols $\Rightarrow$ more economical coding $\Rightarrow$ low bandwidth

Average Codeword Length

Let f_i be the frequency of i-th symbol and ℓ_i be its codeword length. Average code length captures the average-case length of codeword for source (Γ, F)

$$L = \sum_{i=1}^{n} f_i \ell_i$$

Problem. Given a source (Γ, F), finding its optimal encoding that minimizes the average codeword length.

Average Codeword Length

Let f_i be the frequency of i-th symbol and ℓ_i be its codeword length. Average code length captures the average-case length of codeword for source (Γ, F)

$$L = \sum_{i=1}^{n} f_i \ell_i$$

Problem. Given a source (Γ, F), finding its optimal encoding that minimizes the average codeword length.

Wait... There is still a subtle problem here.



Prefix-free Coding

Prefix-free. No codeword can be a prefix of another codeword.

prefix code (前缀码) $\rightsquigarrow$ prefix-free code

Prefix-free Coding

Prefix-free. No codeword can be a prefix of another codeword.

prefix code (前缀码) $\rightsquigarrow$ prefix-free code

All fixed-length encodings naturally satisfy prefix-free property.

- Prefix-free property is only meaningful for variable-length encoding.

Prefix-free Coding

Prefix-free. No codeword can be a prefix of another codeword.

prefix code (前缀码) $\rightsquigarrow$ prefix-free code

All fixed-length encodings naturally satisfy prefix-free property.

- Prefix-free property is only meaningful for variable-length encoding.

Why prefix-free encoding is useful (consider the case of binary encoding)

- Not prefix-free: the coding may not be uniquely decipherable. Out-of-band channel is needed to transmit delimiter.
- Prefix-free: uniquely decipherable, decoding does not require out-of-band transmit

Ambiguity of Non-Prefix-Free Encoding

Example. Non prefix-free encoding

- $\langle a, 001 \rangle$, $\langle b, 00 \rangle$, $\langle c, 010 \rangle$, $\langle d, 01 \rangle$

Decoding of string like 0100001 is ambiguous

- decoding 1: $01 \mid 00 \mid 001 \Rightarrow (d, b, a)$
- decoding 2: $010 \mid 00 \mid 01 \Rightarrow (c, b, d)$

Ambiguity of Non-Prefix-Free Encoding

Example. Non prefix-free encoding

- $\langle a, 001\rangle$, $\langle b, 00\rangle$, $\langle c, 010\rangle$, $\langle d, 01\rangle$

Decoding of string like 0100001 is ambiguous

- decoding 1: $01 \mid 00 \mid 001 \Rightarrow (d, b, a)$
- decoding 2: $010 \mid 00 \mid 01 \Rightarrow (c, b, d)$

Refinement of Problem

How to find an optimal prefix-free encoding?

Tree Representation of Prefix-free Encoding

Any prefix-free encoding can be represented by a binary tree.

- all symbols appear at leaves
  - this property implies prefix-freeness, since no leaf node could be a prefix of other leaf's nodes
- each codeword is generated by a path from root to leaf, interpreting "left" as 0 and "right" as 1
- codeword length ℓ_i for symbol $i \leftarrow$ depth d_i of leaf node i in the tree

Tree Representation of Prefix-free Encoding

Any prefix-free encoding can be represented by a binary tree.

- all symbols appear at leaves
  - this property implies prefix-freeness, since no leaf node could be a prefix of other leaf's nodes
- each codeword is generated by a path from root to leaf, interpreting "left" as 0 and "right" as 1
- codeword length ℓ_i for symbol $i \leftarrow$ depth d_i of leaf node i in the tree

Bonus: Decoding is unique

- A string of bits is decoded by starting at the root, reading the string from left to right to move downward along the tree.
- Whenever a leaf is reached, outputting the corresponding symbol and returning to the root.

Simple Fact

An optimal prefix-free encoding corresponds to a *full binary tree*

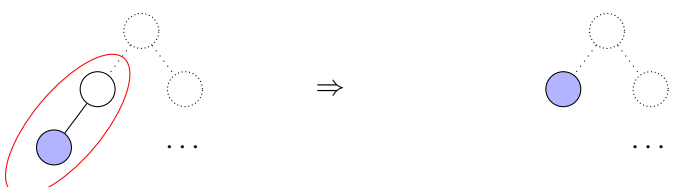
- every node has either zero or two children
- except leaves, each internal node has two children

Simple Fact

An optimal prefix-free encoding corresponds to a *full binary tree*

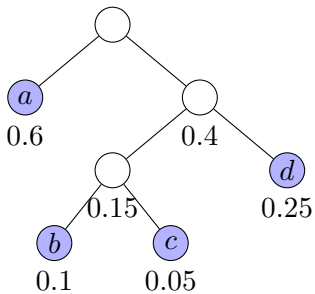
- every node has either zero or two children
- except leaves, each internal node has two children

Proof. If not, there must be a local structure as shown in the picture on the right, we can remove the node with only one child.



An Example of Tree Representation (1/2)

Example. $\langle a, 0.6\rangle, \langle b, 0.05\rangle, \langle c, 0.1\rangle, \langle d, 0.25\rangle$



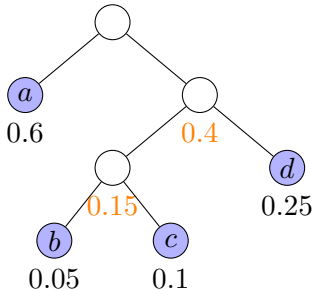
$$(0.05 + 0.1) \times 3 + 0.25 \times 2 + 0.6 \times 1$$
$$= 1.55$$

cost of tree : $L(T)$

depth of i-th symbol in the tree : d_i

$$L(T) = \sum_{i=1}^{n} f_i \cdot d_i = \sum_{i=1}^{n} f_i \cdot \ell_i = L(\Gamma)$$

An Example of Tree Representation (2/2)



$$(0.05 + 0.1) + 0.15 + 0.25 + 0.4 + 0.6 = 1.55$$

Define the frequency of an *internal* node as the sum of the frequences of its two children. The cost of a tree is the sum of the frequencies of all leaves and internal nodes, except the root.

- for full binary tree with $n > 1$ leaves, there is one root node, $n - 2$ internal nodes

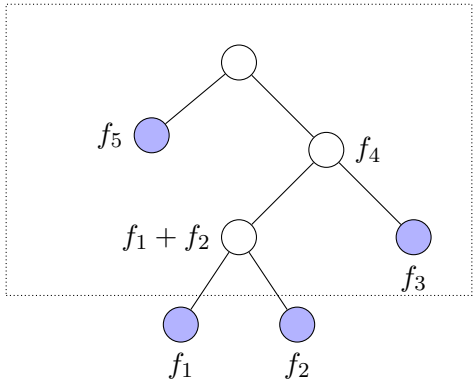
Another way to write the cost function:

$$L(T) = \sum_{i=1}^{2n-2} f_i$$

Greedy Algorithm of Building Optimal Full Binary Tree

Constructing the optimal full binary tree in a greedy manner:

1. find the two symbols with the smallest frequencies, say 1 and 2
2. make them the children of a new node, which then has frequency $f_i + f_j$
3. remove f_1 and f_2 from the list of frequencies, insert $(f_1 + f_2)$, then iterate to repeat the above steps.



Huffman Coding

[David A. Huffman, 1952] A Method for the Construction of Minimum-Redundancy Codes.

- Huffman coding is a particular type of optimal prefix-free code that is commonly used for lossless data compression.

Huffman Coding

[David A. Huffman, 1952] A Method for the Construction of Minimum-Redundancy Codes.

- Huffman coding is a particular type of optimal prefix-free code that is commonly used for lossless data compression.

Huffman's method can be efficiently implemented, finding a code in time linear to the number of input weights if these weights are sorted.

Huffman Coding

[David A. Huffman, 1952] A Method for the Construction of Minimum-Redundancy Codes.

- Huffman coding is a particular type of optimal prefix-free code that is commonly used for lossless data compression.

Huffman's method can be efficiently implemented, finding a code in time linear to the number of input weights if these weights are sorted.

Shannon's source coding theorem

- Shannon entropy is a lower bound on the smallest codeword length

$$\tilde{\mathrm{H}}(\Gamma) = \sum_{i=1}^{n} f_i \log \frac{1}{f_i}$$

Huffman Coding

[David A. Huffman, 1952] A Method for the Construction of Minimum-Redundancy Codes.

- Huffman coding is a particular type of optimal prefix-free code that is commonly used for lossless data compression.

Huffman's method can be efficiently implemented, finding a code in time linear to the number of input weights if these weights are sorted.

Shannon's source coding theorem

- Shannon entropy is a lower bound on the smallest codeword length

$$\tilde{\mathrm{H}}(\Gamma) = \sum_{i=1}^{n} f_i \log \frac{1}{f_i}$$

Huffman coding is very close to the theoretical limit established by Shannon.

History of Huffman Coding

Huffman coding remains in wide use because of its simplicity, high speed, and lack of patent coverage.

- It is often used as a "back-end" to other compression methods.
- DEFLATE (PKZIP's algorithm) and multimedia codes such as JPEG and MP3 have a front-end model and quantization followed by the use of prefix-free codes

History of Huffman Coding

Huffman coding remains in wide use because of its simplicity, high speed, and lack of patent coverage.

- It is often used as a "back-end" to other compression methods.
- DEFLATE (PKZIP's algorithm) and multimedia codes such as JPEG and MP3 have a front-end model and quantization followed by the use of prefix-free codes

In 1951, David A. Huffman and his MIT information theory classmates were given the choice of a term paper or a final exam. The professor, Robert M. Fano, assigned a term paper on the problem of finding the most efficient binary code. Huffman, unable to prove any codes were the most efficient, was about to give up and start studying for the final when he hit upon the idea of using a frequency-sorted binary tree and quickly proved this method the most efficient.
In doing so, Huffman outdid Fano, who had worked with information theory inventor Claude Shannon to develop a similar code. Building the tree from the bottom up guaranteed optimality, unlike top-down Shannon-Fano coding.

Priority Queue

Priority queue: an abstract data type where each element has an associated priority. and the element with the highest (or lowest) priority is always served first, regardless of insertion order. Heaps are the most efficient way to implement them. It maintains a set of elements (a.k.a. keys) with associated numeric values (representing the priority) and supports the following operations.

- Make-queue. Build a priority queue out of the given elements, with the given priority values.
- Insert. Add a new element-value pair to the set.
- Decrease-key. Adjust an element's priority.
- Extract-min. Return the element with smallest value, and remove it from the set.

Pseudocode of Huffman Coding

Algorithm 1: HuffmanEncoding($S = \{i\}, 0 \leq f(i) \leq 1$)

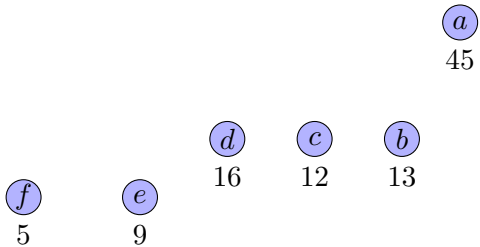
Output: An encoding tree with n leaves

let Q be a priority queue of integers (symbol index), ordered by frequency;
for $i = 1$ *to* n **do** insert(Q, i);
for $k = n + 1$ *to* $2n - 1$ **do**
    i = extractmin(Q), j = extractmin(Q);
    create a node numbered k with children i, j;
    $f(k) \leftarrow f(i) + f(j)$; `// i is left child, j is right child`
    record tuple (i, j, k);
    insert(Q, k);
end

- After each operation, the length of queue decreases by 1. When there is only one element left, the construction of Huffman tree finishes.

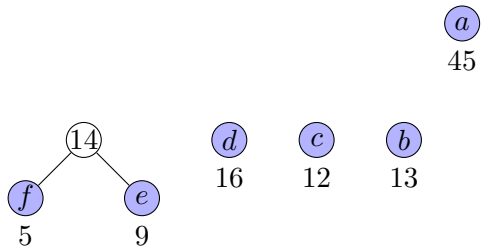
Demo of Huffman Encoding

Input: $\langle a, 0.45\rangle, \langle b, 0.13\rangle, \langle c, 0.12\rangle, \langle d, 0.16\rangle, \langle e, 0.09\rangle, \langle f, 0.05\rangle$



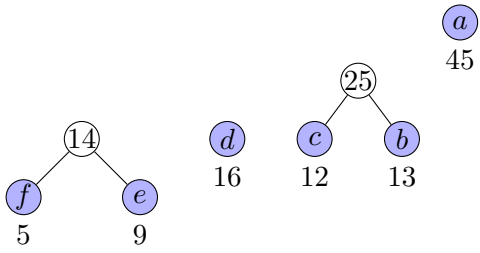
Demo of Huffman Encoding

Input: $\langle a, 0.45\rangle, \langle b, 0.13\rangle, \langle c, 0.12\rangle, \langle d, 0.16\rangle, \langle e, 0.09\rangle, \langle f, 0.05\rangle$



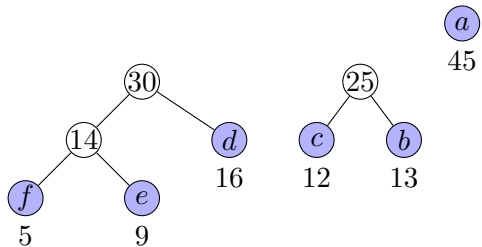
Demo of Huffman Encoding

Input: $\langle a, 0.45\rangle, \langle b, 0.13\rangle, \langle c, 0.12\rangle, \langle d, 0.16\rangle, \langle e, 0.09\rangle, \langle f, 0.05\rangle$



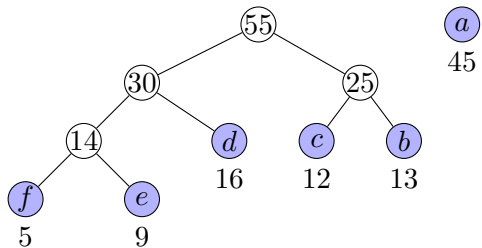
Demo of Huffman Encoding

Input: $\langle a, 0.45\rangle, \langle b, 0.13\rangle, \langle c, 0.12\rangle, \langle d, 0.16\rangle, \langle e, 0.09\rangle, \langle f, 0.05\rangle$



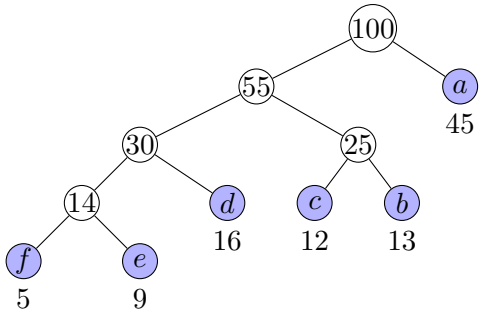
Demo of Huffman Encoding

Input: $\langle a, 0.45\rangle, \langle b, 0.13\rangle, \langle c, 0.12\rangle, \langle d, 0.16\rangle, \langle e, 0.09\rangle, \langle f, 0.05\rangle$



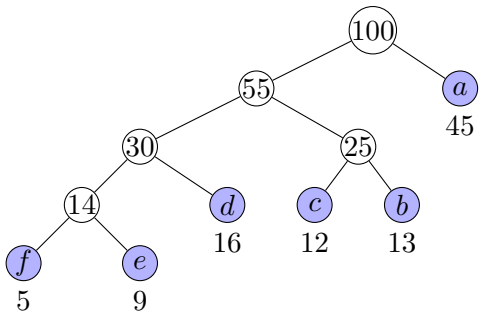
Demo of Huffman Encoding

Input: $\langle a, 0.45\rangle, \langle b, 0.13\rangle, \langle c, 0.12\rangle, \langle d, 0.16\rangle, \langle e, 0.09\rangle, \langle f, 0.05\rangle$



Demo of Huffman Encoding

Input: $\langle a, 0.45\rangle, \langle b, 0.13\rangle, \langle c, 0.12\rangle, \langle d, 0.16\rangle, \langle e, 0.09\rangle, \langle f, 0.05\rangle$



Encoding: $\langle f, 0000\rangle, \langle e, 0001\rangle, \langle d, 001\rangle, \langle c, 010\rangle, \langle b, 011\rangle, \langle a, 1\rangle$

Average code length: $4 \times (0.05 + 0.09) + 3 \times (0.16 + 0.12 + 0.13) + 1 \times 0.45 = 2.24$

Property of Optimal Prefix-free Encoding: Lemma 1

Lemma 1. Let x and y be two symbols in Γ with the smallest frequencies, then there must exist an optimal full binary tree has x and y as sibling leaves at the deepest level.

Property of Optimal Prefix-free Encoding: Lemma 1

Lemma 1. Let x and y be two symbols in Γ with the smallest frequencies, then there must exist an optimal full binary tree has x and y as sibling leaves at the deepest level.

Proof sketch

- Breaking the lemma into cases depending on $|\Gamma|$.
- By the correspondence between encoding scheme and tree, just need to prove the tree T stated by lemma is optimal than trees T' of other forms.

Property of Optimal Prefix-free Encoding: Lemma 1

Lemma 1. Let x and y be two symbols in Γ with the smallest frequencies, then there must exist an optimal full binary tree has x and y as sibling leaves at the deepest level.

Proof sketch

- Breaking the lemma into cases depending on $|\Gamma|$.
- By the correspondence between encoding scheme and tree, just need to prove the tree T stated by lemma is optimal than trees T' of other forms.

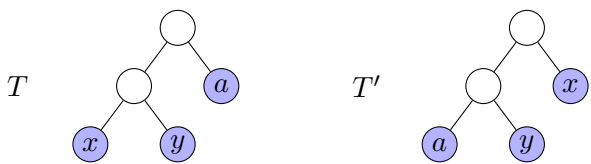
Case $|\Gamma| = 1$, the lemma obviously holds. Only one possibility: one root node.

Case $|\Gamma| = 2$, the lemma obviously holds. Only one possibility: one-level full binary tree with two leaves.

Proof of Lemma 1 (continue)

Case $|\Gamma| = 3$, two possibilities T and T'

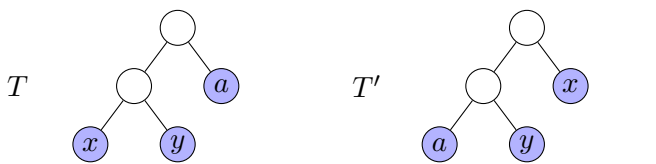
- T: both x, y are sibling leaves in the second level (by algorithm)
- T': only one of x and y is a leaf node in the second level



Proof of Lemma 1 (continue)

Case $|\Gamma| = 3$, two possibilities T and T'

- T: both x, y are sibling leaves in the second level (by algorithm)
- T': only one of x and y is a leaf node in the second level

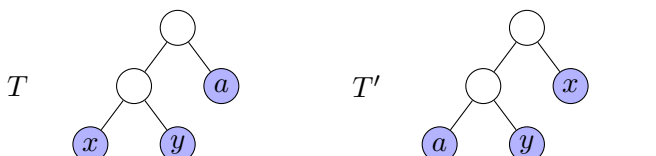


$$L(T') - L(T) = f_a \times 2 + f_x - (f_x \times 2 + f_a) = f_a - f_x \geq 0$$

Proof of Lemma 1 (continue)

Case $|\Gamma| = 3$, two possibilities T and T'

- T: both x, y are sibling leaves in the second level (by algorithm)
- T': only one of x and y is a leaf node in the second level



$$L(T') - L(T) = f_a \times 2 + f_x - (f_x \times 2 + f_a) = f_a - f_x \geq 0$$

$$\Rightarrow T \text{ is optimal}$$

Proof of Lemma 1 (continue)

Case $|\Gamma| \geq 4$, consider the following possible sub-cases:

- both x and y are not sibling leaf nodes in the deepest level: use (x, y) to replace sibling leaf nodes (a, b) in the deepest level (such (a, b) must exist) to obtain T

$$\begin{aligned} &L(T') - L(T) = \\ =&f_a d_a + f_b d_b + f_x d_x + f_y d_y - (f_x d_a + f_y d_b + f_a d_x + f_b d_y) \\ =&(d_a - d_x)(f_a - f_x) + (d_b - d_y)(f_b - f_y) \geq 0 \end{aligned}$$

- x and y are in the deepest level but not sibling nodes: simple swap to obtain T':

$$L(T) = L(T')$$

- only one of x and y in the deepest level, w.l.o.g. assume x is in the deepest level and its sibling is a, swap y and a to obtain T. Similar to $|\Gamma| = 3$, we have:

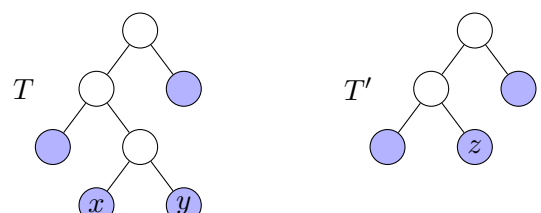
$$L(T') - L(T) \geq 0$$

Properties of Optimal Prefix-free Encoding: Lemma 2

Lemma 2. Let T be the prefix-free encoding full binary tree for (Γ, F), and x and y be two sibling leaf nodes and z be their parent node. Define T' as the prefix-free encoding full binary tree for $\Gamma' = (\Gamma \setminus \{x, y\}) \cup \{z\}$ derived from T, where $f'_c = f_c$ for all $c \in \Gamma \setminus \{x, y\}$, $f_z = f_x + f_y$. Then:

T' is a full binary tree with cost

$$L(T) = L(T') + f_x + f_y$$



Proof of Lemma 2

$\forall c \in \Gamma - \{x, y\}$, we have $d_c = d'_c \Rightarrow$

$$\begin{cases} f_c d_c = f_c d'_c \\ d_x = d_y = d'_z + 1 \\ \Gamma - \{x, y\} = \Gamma' - \{z\} \end{cases}$$

$$\begin{aligned} L(T) =& \sum_{c \in \Gamma} f_c d_c = \left(\sum_{c \in \Gamma - \{x,y\}} f_c d_c \right) + (f_x d_x + f_y d_y) \\ =& \left(\sum_{c \in \Gamma' - \{z\}} f_c d'_c \right) + f_z d'_z + (f_x + f_y) \\ =& L(T') + f_x + f_y \end{aligned}$$

Correctness Proof of Huffman Encoding

Theorem. Huffman algorithm yields optimal prefix-free encoding binary tree for all $|\Gamma| \geq 2$.

Proof. Mathematical induction

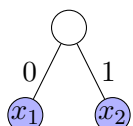
Induction basis. For $|\Gamma| = 2$, Huffman algorithm yields optimal prefix-free encoding.

Induction step. Assume Huffman algorithm encoding yields optimal prefix-free encoding for size k, then it also yields optimal prefix-free encoding for size $k + 1$.

Induction Basis

$k = 2$, $\Gamma = \{x_1, x_2\}$

Every codeword at least require at least one bit. Huffman algorithm yields codeword 0 and 1, which is optimal prefix-free encoding.



Induction Step (1/3)

Assume Huffman algorithm yields optimal prefix-free encoding for input size k. Now, consider input size $k+1$, a.k.a. $|\Gamma| = k+1$

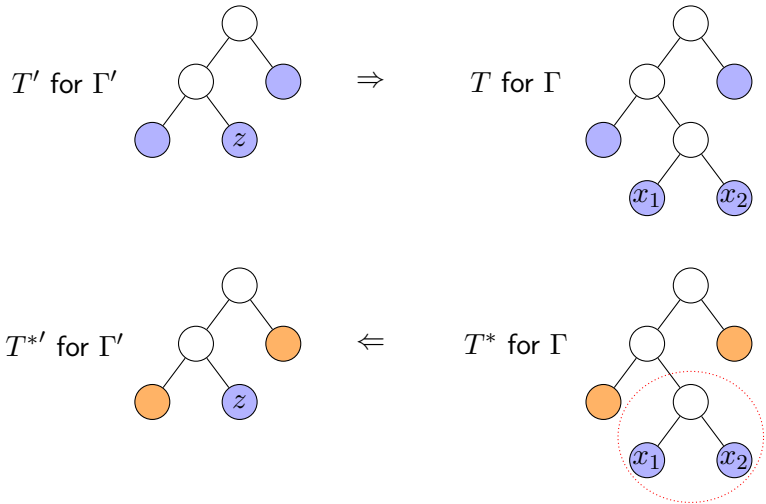
$$\Gamma = \{x_1, x_2, \ldots, x_{k+1}\}$$

Let $\Gamma' = (\Gamma - \{x_1, x_2\}) \cup \{z\}$, $f_z = f_{x_1} + f_{x_2}$

Induction hypothesis $\Rightarrow$ Huffman algorithm generates an optimal prefix-free encoding tree T' for Γ', where frequencies are f_z and f_{x_i} $(i = 3, 4, \ldots, k+1)$.

Induction Step (2/3)

Claim. Append (x_1, x_2) as z's children to T', obtaining T (greedy algorithm's output), which is an optimal prefix-free encoding tree for $\Gamma = (\Gamma' - \{z\}) \cup \{x_1, x_2\}$.



Induction Step (3/3)

Proof. If not, then there exists an optimal prefix-free encoding tree T^* that has x_1 and x_2 as the sibling leaves in the deepest level (existence is guaranteed by Lemma 1), and $L(T^*) < L(T)$.

Idea. Reduce to the optimality of T' for Γ'

Remove x_1 and x_2 from T^*, obtaining a new encoding tree $T^{*\prime}$ for Γ'. Lemma 2 $\Rightarrow$

$$\begin{aligned} L(T^{*\prime}) &= L(T^*) - (f_{x_1} + f_{x_2}) \\ &< L(T) - (f_{x_1} + f_{x_2}) \\ &= L(T') \end{aligned}$$

This contradicts the premise that T' is an optimal prefix-free encoding tree for Γ'.

Application: Files Merge

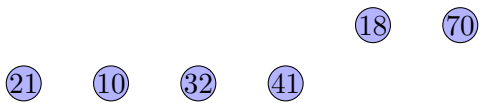
Problem. Given n sorted files $\Gamma = \{1, \ldots, n\}$, where file i contains f_i items. Now, the task is merging them into a one sorted file using two-way merges.

Represent the merging process as a bottom-up binary tree.

- leaf nodes: files labeled with $\{1, \ldots, n\}$
- parent node of i and j: value is $f_i + f_j$ (number of items after merging)

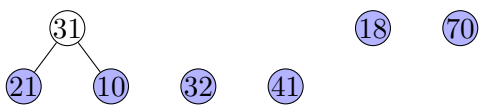
Demo of 2-ary Sequential Merge

Example. $\Gamma = \{21, 10, 32, 41, 18, 70\}$



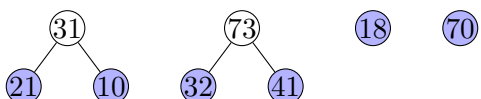
Demo of 2-ary Sequential Merge

Example. $\Gamma = \{21, 10, 32, 41, 18, 70\}$



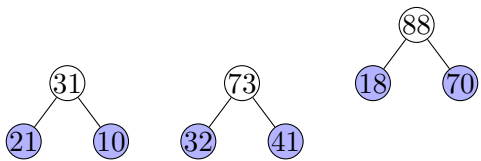
Demo of 2-ary Sequential Merge

Example. $\Gamma = \{21, 10, 32, 41, 18, 70\}$



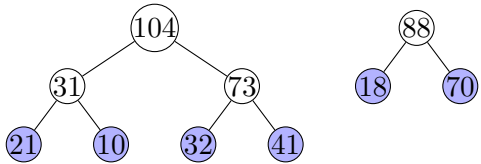
Demo of 2-ary Sequential Merge

Example. $\Gamma = \{21, 10, 32, 41, 18, 70\}$



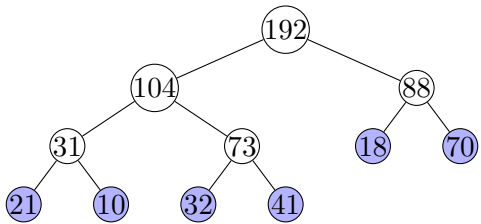
Demo of 2-ary Sequential Merge

Example. $\Gamma = \{21, 10, 32, 41, 18, 70\}$



Demo of 2-ary Sequential Merge

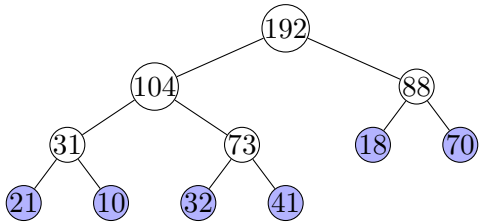
Example. $\Gamma = \{21, 10, 32, 41, 18, 70\}$



Complexity of 2-way Merge (1/2)

Worst-case complexity of merging ordered $A[k]$ and $B[l]$ into ordered $C[k+l]$

- same to merge operation in MergeSort
- $W(k,l) \leq k+l-1$



Bottom-up calculation of worst-case merging complexity:

$$(21+10-1)+(32+41-1)+(18+70-1)$$
$$+(31+73-1)+(104+88-1)=483$$

Complexity of 2-way Merge (2/2)

Calculation from n leaf nodes

$$(21 + 10 + 32 + 41) \times 3 + (18 + 70) \times 2 - 5 = 483$$

Worst-case complexity

$$W(n) = \left(\sum_{i=1}^{n} f_i d_i \right) - (n - 1)$$

Complexity of 2-way Merge (2/2)

Calculation from n leaf nodes

$$(21 + 10 + 32 + 41) \times 3 + (18 + 70) \times 2 - 5 = 483$$

Worst-case complexity

$$W(n) = \left(\sum_{i=1}^{n} f_i d_i \right) - (n - 1)$$

How to prove the correctness of formula?

Complexity of 2-way Merge (2/2)

Calculation from n leaf nodes

$$(21 + 10 + 32 + 41) \times 3 + (18 + 70) \times 2 - 5 = 483$$

Worst-case complexity

$$W(n) = \left(\sum_{i=1}^{n} f_i d_i \right) - (n - 1)$$

How to prove the correctness of formula?

- The tree is generated in bottom-up manner, and must be a full binary tree. Each non-leaf node corresponds to a merge operation, contribution to $W(n)$ is -1.
- The number of internal nodes $= m$. Except leaf nodes, for all internal nodes: in-degree $= 1$, out-degree $= 2$.

$$\sum \text{out-degree} = 2m, \sum \text{in-degree} = n + (m - 1) \Rightarrow m = n - 1$$

Optimal File Merge

Goal. Find a merge sequence that minimizes $W(n)$

- The is the same problem in spirit as Huffman encoding (except a fixed constant $n-1$).

Solution. Treat the item numbers as frequency, apply Huffman algorithm to generate the merging tree.

Special example. $n = 2^k$ files and each file has the same number of items. In this case, Huffman algorithm generates a perfect binary tree, which is same as iterated version 2-way merge sort.

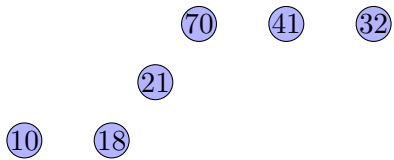
- This also demonstrates the optimality of MergeSort.

Demo of Huffman Tree Merging

Input. $\Gamma = \{21, 10, 32, 41, 18, 70\}$

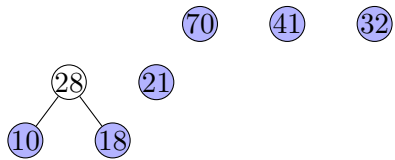
Demo of Huffman Tree Merging

Input. $\Gamma = \{21, 10, 32, 41, 18, 70\}$



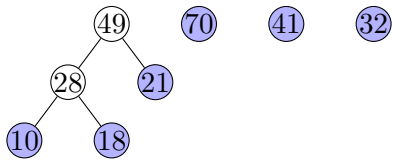
Demo of Huffman Tree Merging

Input. $\Gamma = \{21, 10, 32, 41, 18, 70\}$



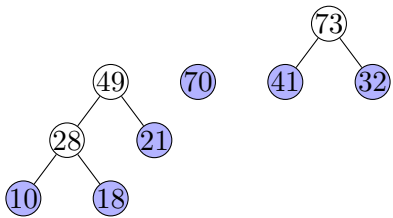
Demo of Huffman Tree Merging

Input. $\Gamma = \{21, 10, 32, 41, 18, 70\}$



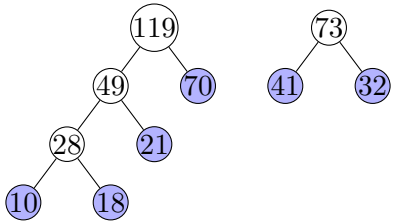
Demo of Huffman Tree Merging

Input. $\Gamma = \{21, 10, 32, 41, 18, 70\}$



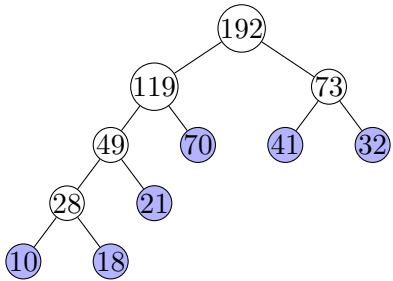
Demo of Huffman Tree Merging

Input. $\Gamma = \{21, 10, 32, 41, 18, 70\}$



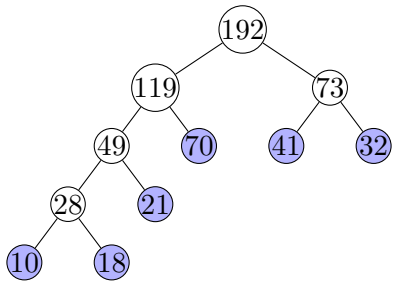
Demo of Huffman Tree Merging

Input. $\Gamma = \{21, 10, 32, 41, 18, 70\}$



Demo of Huffman Tree Merging

Input. $\Gamma = \{21, 10, 32, 41, 18, 70\}$



Cost. $(10 + 18) \times 4 + 21 \times 3 + (70 + 41 + 32) \times 2 - 5 = 456 < 483$

Recap of Huffman Coding

Refine the problem as optimal prefix-free encoding

Model the encoding scheme as building an optimal full binary tree

- Objective function: the cost of tree

Prove the greedy construction is optimal

- lemma 1: show the optimal encoding tree must exhibit certain local structure
- lemma 2: establish the connection between instance of k and $k+1$
- theorem combines lemma 1 and lemma 2 to prove
  - optimal for $k \Rightarrow$ optimal for $k+1$

Outline

1. Huffman Coding

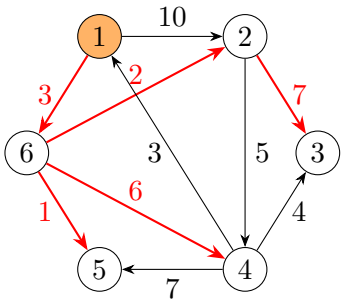
2. Single Source Shortest Path Problem
   - Dijkstra's Algorithm

3. Minimum Spanning Tree
   - The Cut-Based MST Algorithm
   - Kruskal's Algorithm
   - Prim's Algorithm

Single Source Shortest Path (SSSP) Problem

Problem. Given a directed graph $G = (V, E)$ with edge weight $e(i, j) \geq 0$, find the shortest path from a source node $s \in V$ to every vertex in G.





$$d(1,2) = 5 : \langle 1 \to 6 \to 2 \rangle$$
$$d(1,3) = 12 : \langle 1 \to 6 \to 2 \to 3 \rangle$$
$$d(1,4) = 9 : \langle 1 \to 6 \to 4 \rangle$$
$$d(1,5) = 4 : \langle 1 \to 6 \to 5 \rangle$$
$$d(1,6) = 3 : \langle 1 \to 6 \rangle$$

Applications

The problem has many practical applications

- Map routing.
- Seam carving.
- Robot navigation.
- Texture mapping.
- Typesetting in LaTeX.
- Urban traffic planning.
- Telemarketer operator scheduling.
- Routing of telecommunications messages.
- Network routing protocols (OSPF, BGP, RIP).
- Optimal truck routing through given traffic congestion pattern.

Outline

1. Huffman Coding

2. Single Source Shortest Path Problem
   - Dijkstra's Algorithm

3. Minimum Spanning Tree
   - The Cut-Based MST Algorithm
   - Kruskal's Algorithm
   - Prim's Algorithm

A Brief Biography of Dijkstra



Figure: Edsger Wybe Dijkstra: a Dutch computer scientist, programmer, software engineer, science essayist.

A Brief Biography of Dijkstra



Figure: Edsger Wybe Dijkstra: a Dutch computer scientist, programmer, software engineer, science essayist.

- dreamed of becoming a representative of the United Nations
- straight A student in high school: encouraged to pursue science
- major in theoretical physics in university: electronic computer appeared at that time, and his father sent him to attend a programming course in Cambridge university.
- devote himself to programming: more intellectual challenge, ruthless cause even a single bit error is not permitted

Dijkstra's Achievements

One of the most influential figures of computing science, shaped the new discipline both as an engineer and a theorist.

- In 1972, he became the first person who was neither American nor British to win the Turing Award.

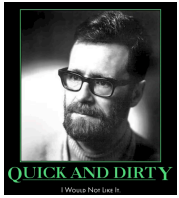
Dijkstra's Achievements

One of the most influential figures of computing science, shaped the new discipline both as an engineer and a theorist.

- In 1972, he became the first person who was neither American nor British to win the Turing Award.

His fundamental contributions cover diverse areas of CS:

- algorithm, compiler, operating system, distributed computing (PODC), programming language, programming paradigm and methodology, programming verification, graph theory ...
- pioneer many brand new research areas (to list some standard notions: mutex, deadlock, semaphore)





Dijkstra's Style

Dijkstra chose no reference style to preserve his self-reliance.

His approach to teaching was unconventional

- A quick and deep thinker while engaged in the act of lecturing
- Never followed a textbook, with the possible exception of his own while it was under preparation.
- Assigned challenging homework problems, and would study his students' solutions thoroughly.
- Conducted his final examinations orally, over a whole week. Each student was examined in Dijkstra's office or home, and an exam lasted several hours.

He invented many computer techniques but rarely used computers.

Story about Dijkstra's Algorithm

Dijkstra had to build a demo to promote ARMAC (an early electronic computer).

- think over it in cafe: finding shortest path between two cities in Netherlands could be a good problem
- design the algorithm in 20 minutes (perhaps the greatest achievement in his career)
- published 3 years later widely used today, but not acknowledged by mathematicians at that time

According to later interview, this algorithm is so simple because there are no pen and paper in cafe: forced him to avoid complicated design but pursue simplicity instead.

> Most of Dijkstra's work is simple, efficient and elegant, originated from his mother's guidance. The scarcity of resources often stimulates the best creativity.

Dijkstra's Algorithm

Intuition. (Best-First Search) Explore the unknown world step by step, the cognition on each step is correct.

Greedy approach. Maintain a set of explored nodes S for which algorithm has determined the shortest path distance from s^*, as well as a set of unexplored nodes U.

1. Initialize $S = \emptyset$, $d(s^*, s^*) = 0$, $d(s^*, v) = \infty$; $U = V$
2. Repeatedly choose unexplored node $u \in U$ with

$$\min_{u \in U} d(s^*, u)$$

   - add u to S
   - for the rest nodes $v \in U$, update $d(s^*, v) = d(s^*, u) + e(u, v)$ if the right part is shorter and set $\text{prev}(v) = u$

3. Finishes when $S = V$, a.k.a. $U = \emptyset$.

Implementation Details

Data structure:

- s^*: source point
- S: nodes have been explored
- U: nodes have not been explored
- $d(s^*, v)$: the length of current shortest path from s^* to v. If $v \in S$, then it is the final length of shortest path.
- prev(v): preceding node of v, used to track path

Critical data structure. Priority queue $\Rightarrow$ tracking u s.t. $\min_{u \in U} d(s^*, u)$

- Suppose u is added to S and there is an edge (u, v) from u to $v \in U$. Then, it suffices to update:

$$d(s^*, v) = \min\{d(s^*, v), d(s^*, u) + e(u, v)\}$$

  - Thus, for each $v \notin S$, $d(s^*, v)$ can only decrease since S only increases and includes more u's.

Dijkstra's Algorithm to SSSP

Algorithm 2: Dijkstra$(G = (V, E), s^*)$

1: $S = \emptyset$, $d(s^*, s^*) = 0$, $U = V$;
2: **for** $u \in U - \{s^*\}$ **do** $d(s^*, u) = \infty$, prev$(u) = \perp$;
3: $Q \leftarrow$ makequeue(V, d) ; `// using dist-values as values`
4: **while** $S \neq V$ **do**
5: $u =$ extractmin(Q), $S = S \cup \{u\}$, $U = U - \{u\}$;
6: **for** *all* $v \in U$ **do**
7: **if** $d(s^*, v) > d(s^*, u) + e(u, v)$ **then** $d(s^*, v) = d(s^*, u) + e(u, v)$;
`// update`
8: prev$(v) = u$; decreasekey(Q, v) ;
9: **end**
10: **end**

- Dijkstra's algorithm works correctly for both directed and undirected graphs provided that there are no negative edges.

Dijkstra's Algorithm: Which Priority Queue?

Cost of priority queue operations. $|V|$ insert, $|V|$ extract-min, $O(|E|)$ decrease-key (explain why)

Performance: depends heavily on priority queue implementation

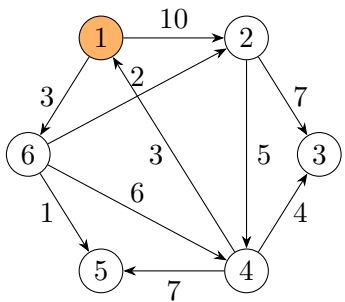
- Array implementation optimal for dense graphs.
- Binary heap much faster for sparse graphs.
- 4-way heap worth the trouble in performance-critical situations.
- Fibonacci/Brodal best in theory, but not worth implementing.

Implementation	insert	extract-min	decrease-key	total cost
unordered array	$O(1)$	$O(\|V\|)$	$O(1)$	$O(\|V\|^2)$
binary heap	$O(\log \|V\|)$	$O(\log \|V\|)$	$O(\log \|V\|)$	$O(\|E\| \log \|V\|)$
d-ary heap	$O(\log_d \|V\|)$	$O(d \log_d \|V\|)$	$O(\log_d \|V\|)$	$O(\|E\| \log_d \|V\| + \|V\| d \log_d \|V\|)$
Fibonacci heap	$O(1)$	$O(\log \|V\|)^\dagger$	$O(1)^\dagger$	$O(\|E\| + \|V\| \log \|V\|)$
Brodal queue	$O(1)$	$O(\log \|V\|)$	$O(1)$	$O(\|E\| + \|V\| \log \|V\|)$

† amortized

Demo of Dijkstra's Algorithm

Input. $G = (V, E)$, $s^* = 1$, $V = \{1, 2, 3, 4, 5, 6\}$



$$S = \emptyset$$

$$d(1,1) = 0,\ \mathsf{prev}(1) = \perp$$

$$d(1,2) = \infty,\ \mathsf{prev}(2) = \perp$$

$$d(1,3) = \infty,\ \mathsf{prev}(3) = \perp$$

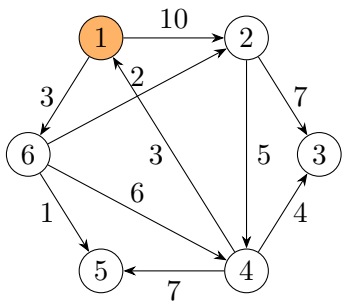
$$d(1,4) = \infty,\ \mathsf{prev}(4) = \perp$$

$$d(1,5) = \infty,\ \mathsf{prev}(5) = \perp$$

$$d(1,6) = \infty,\ \mathsf{prev}(6) = \perp$$

Demo of Dijkstra's Algorithm

Input. $G = (V, E)$, $s^* = 1$, $V = \{1, 2, 3, 4, 5, 6\}$



$$S = \{1\}$$

$$d(1,1) = 0,\ \mathsf{prev}(1) = 1$$

$$d(1,2) = 10,\ \mathsf{prev}(2) = 1$$

$$d(1,6) = 3,\ \mathsf{prev}(6) = 1$$

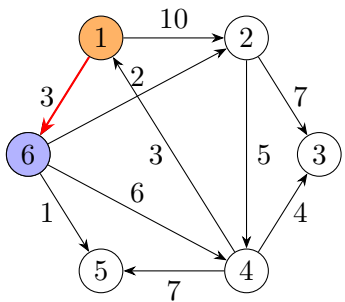
$$d(1,3) = \infty,\ \mathsf{prev}(3) = \perp$$

$$d(1,4) = \infty,\ \mathsf{prev}(4) = \perp$$

$$d(1,5) = \infty,\ \mathsf{prev}(5) = \perp$$

Demo of Dijkstra's Algorithm

Input. $G = (V, E)$, $s^* = 1$, $V = \{1, 2, 3, 4, 5, 6\}$



$$S = \{1, 6\}$$

$$d(1,1) = 0,\ \mathsf{prev}(1) = 1$$

$$d(1,6) = 3,\ \mathsf{prev}(6) = 1$$

$$d(1,2) = 5,\ \mathsf{prev}(2) = 6$$

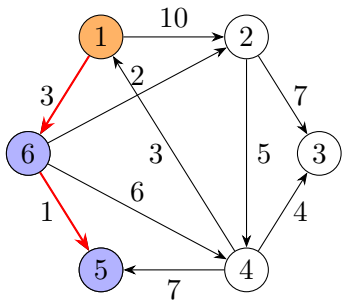
$$d(1,4) = 9,\ \mathsf{prev}(4) = 6$$

$$d(1,5) = 4,\ \mathsf{prev}(5) = 6$$

$$d(1,3) = \infty,\ \mathsf{prev}(3) = \perp$$

Demo of Dijkstra's Algorithm

Input. $G = (V, E)$, $s^* = 1$, $V = \{1, 2, 3, 4, 5, 6\}$



$$S = \{1, 6, 5\}$$

$$d(1,1) = 0,\ \mathsf{prev}(1) = 1$$

$$d(1,6) = 3,\ \mathsf{prev}(6) = 1$$

$$d(1,5) = 4,\ \mathsf{prev}(5) = 6$$

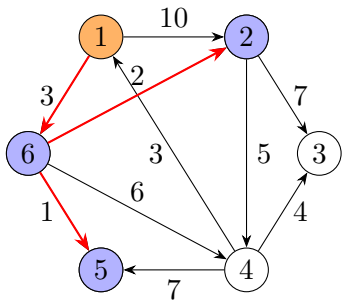
$$d(1,2) = 5,\ \mathsf{prev}(2) = 6$$

$$d(1,4) = 9,\ \mathsf{prev}(4) = 6$$

$$d(1,3) = \infty,\ \mathsf{prev}(3) = \perp$$

Demo of Dijkstra's Algorithm

Input. $G = (V, E)$, $s^* = 1$, $V = \{1, 2, 3, 4, 5, 6\}$



$S = \{1, 6, 5, 2\}$

$d(1, 1) = 0$, $\mathsf{prev}(1) = 1$

$d(1, 6) = 3$, $\mathsf{prev}(6) = 1$

$d(1, 5) = 4$, $\mathsf{prev}(5) = 6$

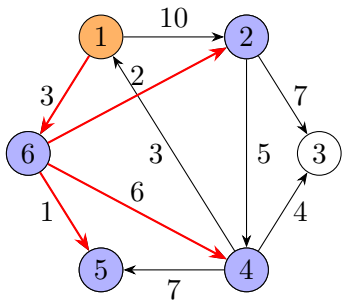
$d(1, 2) = 5$, $\mathsf{prev}(2) = 6$

$d(1, 4) = 9$, $\mathsf{prev}(4) = 6$

$d(1, 3) = 12$, $\mathsf{prev}(3) = 2$

Demo of Dijkstra's Algorithm

Input. $G = (V, E)$, $s^* = 1$, $V = \{1, 2, 3, 4, 5, 6\}$



$S = \{1, 6, 5, 2, 4\}$

$d(1,1) = 0$, $\mathsf{prev}(1) = 1$

$d(1,6) = 3$, $\mathsf{prev}(6) = 1$

$d(1,5) = 4$, $\mathsf{prev}(5) = 6$

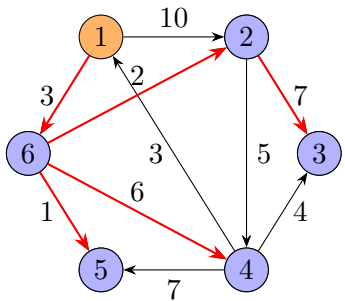
$d(1,2) = 5$, $\mathsf{prev}(2) = 6$

$d(1,4) = 9$ $\mathsf{prev}(4) = 6$

$d(1,3) = 12$, $\mathsf{prev}(3) = 2$

Demo of Dijkstra's Algorithm

Input. $G = (V, E)$, $s^* = 1$, $V = \{1, 2, 3, 4, 5, 6\}$



$S = \{1, 6, 5, 2, 4, 3\}$

$d(1,1) = 0$, $\mathsf{prev}(1) = 1$

$d(1,6) = 3$, $\mathsf{prev}(6) = 1$

$d(1,5) = 4$, $\mathsf{prev}(5) = 6$

$d(1,2) = 5$, $\mathsf{prev}(2) = 6$

$d(1,4) = 9$, $\mathsf{prev}(4) = 6$

$d(1,3) = 12$, $\mathsf{prev}(3) = 2$

Proof of Correctness: Dijkstra's Algorithm (1/2)

Proposition. $|k|$-th step result of Dijkstra's algorithm is always a part of one optimal result. a.k.a. at any time, for each node $u \in S$, $d(s^*, u)$ is the length of the shortest $s^* \rightsquigarrow u$ path.

Proof. By induction on algorithm steps.

Induction basis. $|k| = 1$, $S = \{s^*\}$, $d(s^*, s^*) = 0$. Obviously holds.

Induction step. Assume proposition is true for $k \geq 1$, prove it also holds for $k + 1$.

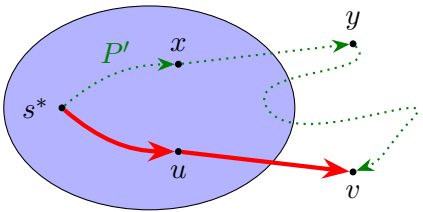
Proof of Correctness: Dijkstra's Algorithm (2/2)

Let v be the next node ($k+1$ step) added to S, u be its predecessor (it must exist), we have $d(s^*, v) = d(s^*, u) + e(u, v)$.

Correctness: Consider any $s^* \rightsquigarrow v$ path P, show $\ell(P) \geq d(s^*, v)$.

- Let (x, y) be the first edge in P that leaves S, and let P' be the subpath to x, then P is already too long as soon as it reaches y. (x may equal u; y may equal v)

$$
\begin{aligned}
\ell(P) &= \ell(P') + e(x,y) + \ell(y,v) && \text{//definition of } P \\
&\geq d(s^*, x) + e(x,y) + 0 && \text{//induction hypothesis} \\
&\geq d(s^*, y) && \text{//definition of } d \\
&\geq d(s^*, v) && \text{//cause Dijkstra chose } v \text{ instead of } y
\end{aligned}
$$



Extensions of Dijkstra's Algorithm

Dijkstra's algorithm and proof extend to several related problems:

- Shortest paths in undirected graphs
- Maximum capacity paths
- Maximum reliability paths

Optimality of Dijkstra's Algorithm

Universal Optimality of Dijkstra via Beyond-Worst-Case Heaps

BERNHARD HAEUPLER, INSAIT, Sofia University "St. Kliment Ohridski", Bulgaria and ETH Zurich, Switzerland
RICHARD HLADÍK, ETH Zurich, Switzerland and INSAIT, Sofia University "St. Kliment Ohridski", Bulgaria
VÁCLAV ROZHOŇ, Charles University, Czech Republic and INSAIT, Sofia University "St. Kliment Ohridski", Bulgaria
ROBERT E. TARJAN, Princeton University, USA
JAKUB TĚTEK, INSAIT, Sofia University "St. Kliment Ohridski", Bulgaria

---

Breaking the Sorting Barrier for Directed Single-Source Shortest Paths

Ran Duan * Jiayi Mao * Xiao Mao † Xinkai Shu ‡ Longhui Yin *

July 31, 2025

Outline

1. Huffman Coding
2. Single Source Shortest Path Problem
   - Dijkstra's Algorithm
3. Minimum Spanning Tree
   - The Cut-Based MST Algorithm
   - Kruskal's Algorithm
   - Prim's Algorithm

Motivation of MST

Real world problem. You are asked to connect a collection of computers by linking them. Each link has a maintenance cost.

What is the cheapest possible network?

Motivation of MST

Real world problem. You are asked to connect a collection of computers by linking them. Each link has a maintenance cost.

What is the cheapest possible network?

This translates into a graph problem:

- nodes – computers
- undirected edges – potential links
- edge's weight – positive maintenance cost

Optimization goal. Pick enough edges so that all the nodes are connected and the total weight is minimal.

What are the properties of these edges?

One immediate observation. Optimal set of edges cannot contain a cycle, since removing an edge from this cycle can reduce the cost without compromising connectivity.

Fact 1. Removing a cycle edge cannot disconnect an undirected graph.

What are the properties of these edges?

One immediate observation. Optimal set of edges cannot contain a cycle, since removing an edge from this cycle can reduce the cost without compromising connectivity.

Fact 1. Removing a cycle edge cannot disconnect an undirected graph.

The solution must be connected and acyclic

- undirected graphs of this kind are called *trees*
- we call the one with minimum total weight as *minimum spanning tree*.

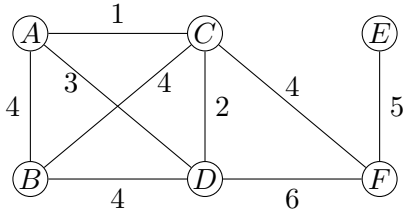
Minimum Spanning Tree

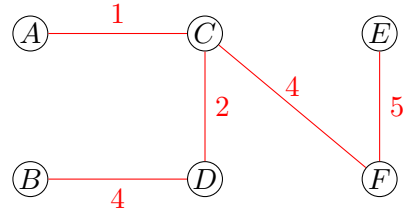
Spanning tree. Let $G = (V, E)$ be an undirected graph, where w_e is the weight of edge e.

- A connected and acyclic subgraph $T = (V, E')$ is called a spanning tree of G.
- The weight of the tree T is weight$(T) = \sum_{e \in E'} w_e$
- The minimum spanning tree is the tree that minimizes weight(T)

The minimum spanning tree may not be unique.

Example of Minimum Spanning Tree





Can you spot another?

Properties of Trees

Definition of Tree. An undirected graph that is connected and acyclic.

Simplicity of structure makes the notion of tree extremely useful

Basic Properties of Trees (relation between nodes and edges)

Fact. A tree with n vertices has $n-1$ edges.

Increase connected nodes: Starting from an empty graph. Initially the n nodes are disconnected from one another.

- First edge: connect two nodes ($n-2$ nodes are left)
- Then: an edge originated from existing connected subgraph adds in, a node is connected

Total edges: $1+(n-2)=n-1$

Decrease separate components: Viewing initially disconnected n nodes as n separate components.

- When $\langle u, v\rangle$ comes up, it merges two components that u and v previously lie in. This step reduces the total number of connected components by 1.
- Over the course of this incremental process, the number of connected components decreases from n to $1 \Rightarrow n-1$ edges must have been added along the way

Property 1 of Tree

Property 1. Any connected, undirected graph $G = (V, E)$ with $|E| = |V| - 1$ is a tree.

Property 1 of Tree

Property 1. Any connected, undirected graph $G = (V, E)$ with $|E| = |V| - 1$ is a tree.

Proof idea. Using definition, just need to show G is acyclic. Suppose it is cyclic, we can run the following iterative procedure to make it acyclic

1. remove one edge from a cycle each time
2. terminates with some graph $G' = (V, E')$, $E' \subseteq E$, which is acyclic.

Operation $\Rightarrow$ G' is still connected $\Rightarrow$ G' is a tree (by def of tree)

Fact $\Rightarrow |E'| = |V| - 1 \Rightarrow E' = E \Rightarrow G' = G$

- No edges were removed, and G was acyclic to start with.

Property 1 of Tree

Property 1. Any connected, undirected graph $G = (V, E)$ with $|E| = |V| - 1$ is a tree.

Proof idea. Using definition, just need to show G is acyclic. Suppose it is cyclic, we can run the following iterative procedure to make it acyclic

1. remove one edge from a cycle each time
2. terminates with some graph $G' = (V, E')$, $E' \subseteq E$, which is acyclic.

Operation $\Rightarrow$ G' is still connected $\Rightarrow$ G' is a tree (by def of tree)

Fact $\Rightarrow$ $|E'| = |V| - 1 \Rightarrow E' = E \Rightarrow G' = G$

- No edges were removed, and G was acyclic to start with.

We can tell whether a connected graph is a tree just by counting how many edges it has.

Property 2 of Tree (Another Characterization)

Property 2. An undirected graph $G = (V, E)$ is a tree if and only if there is a unique path between any pair of nodes.

Property 2 of Tree (Another Characterization)

Property 2. An undirected graph $G = (V, E)$ is a tree if and only if there is a unique path between any pair of nodes.

Forward direction

- In a tree, any two nodes can only have one path between them; for if there were two paths, the union of these paths would contain a cycle.

Property 2 of Tree (Another Characterization)

Property 2. An undirected graph $G = (V, E)$ is a tree if and only if there is a unique path between any pair of nodes.

Forward direction

- In a tree, any two nodes can only have one path between them; for if there were two paths, the union of these paths would contain a cycle.

Backward direction (def: connected+acyclic $\Rightarrow$ tree)

- G has a path between any two nodes $\Rightarrow$ G is connected
- If these paths are unique, then G is acyclic (since any pair of nodes in a cycle have at least two paths between them).

Brief Summary

The above properties give three criteria to decide if an undirected graph is a tree

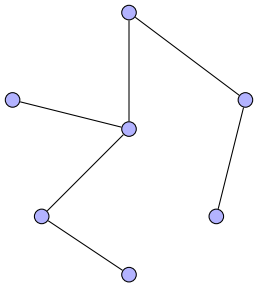
1. Definition: connected and acyclic
2. Property 1: connected and $|V| = |E| + 1$
3. Property 2: there is a unique path between any two nodes

Spanning Tree: Proposition 1

Proposition 1. If T is a spanning tree of G, $e \notin E(T)$, then $T \cup \{e\}$ contains a cycle C.

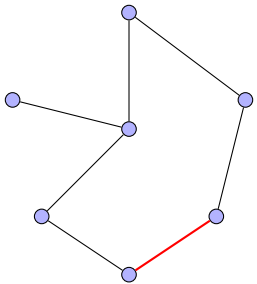
Spanning Tree: Proposition 1

Proposition 1. If T is a spanning tree of G, $e \notin E(T)$, then $T \cup \{e\}$ contains a cycle C.



Spanning Tree: Proposition 1

Proposition 1. If T is a spanning tree of G, $e \notin E(T)$, then $T \cup \{e\}$ contains a cycle C.

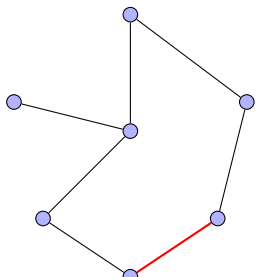


Spanning Tree: Proposition 2

Proposition 2. Removing any edge in the cycle C (in the context of Proposition 1), yields another spanning tree T' of G.

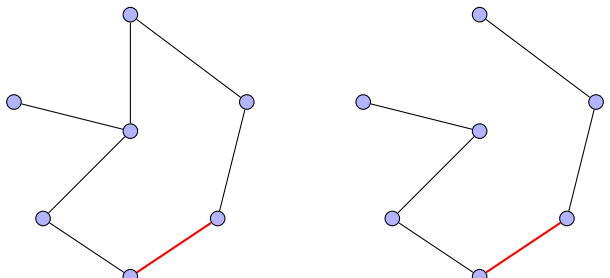
Spanning Tree: Proposition 2

Proposition 2. Removing any edge in the cycle C (in the context of Proposition 1), yields another spanning tree T' of G.



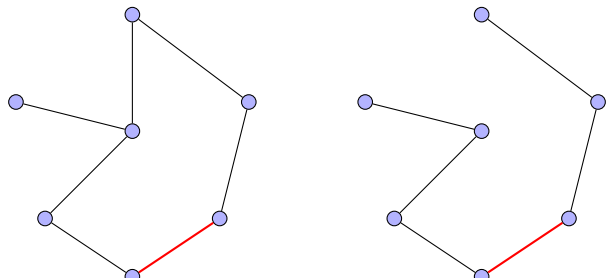
Spanning Tree: Proposition 2

Proposition 2. Removing any edge in the cycle C (in the context of Proposition 1), yields another spanning tree T' of G.



Spanning Tree: Proposition 2

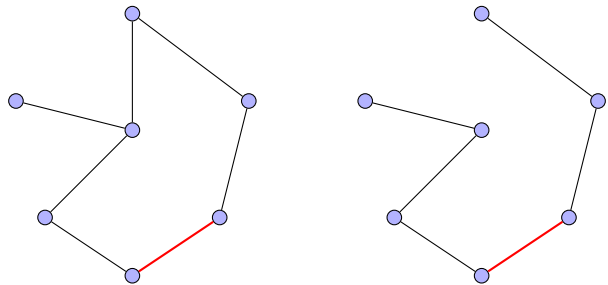
Proposition 2. Removing any edge in the cycle C (in the context of Proposition 1), yields another spanning tree T' of G.



all nodes are still connected + Property 1 $\Rightarrow$ Proposition 2

Spanning Tree: Proposition 2

Proposition 2. Removing any edge in the cycle C (in the context of Proposition 1), yields another spanning tree T' of G.



all nodes are still connected + Property 1 $\Rightarrow$ Proposition 2

This proposition gives a method of creating a new spanning tree from an existing spanning tree.

Applications of MST

The MST problem is fundamental with diverse applications.

- Dithering
- Cluster analysis
- Max bottleneck paths
- Real-time face verification
- LDPC codes for error correction
- Image registration with Renyi entropy
- Find road networks in satellite and aerial imagery
- Reducing data storage in sequencing amino acids in a protein
- Model locality of particle interactions in turbulent fluid flows
- Network design (communication, electrical, computer, road)
- Approximation algorithms for $\mathcal{NP}$-hard problems (e.g., TSP, Steiner tree)

Outline

1. Huffman Coding
2. Single Source Shortest Path Problem
   - Dijkstra's Algorithm
3. Minimum Spanning Tree
   - The Cut-Based MST Algorithm
   - Kruskal's Algorithm
   - Prim's Algorithm

The Cut Property

Say that in the process of building MST, we have already chosen some edges and are so far on the right track.

Which edge should we add next?

If there is a correct strategy, then we can solve MST iteratively. The following lemma gives us a lot of flexibility in our choice.

The Cut Property

Say that in the process of building MST, we have already chosen some edges and are so far on the right track.

Which edge should we add next?

If there is a correct strategy, then we can solve MST iteratively. The following lemma gives us a lot of flexibility in our choice.

A cut of V is a partition of V, say, $(S, V - S)$. A cut is compatible with a set of edges X if no edge in X crosses between S and $V - S$.

Cut property. Suppose the set of edges X is part of a MST of $G = (V, E)$. Let $(S, V - S)$ be any cut compatible with X, the lightest edge e crossing the cut is safe to add. i.e., $X \cup \{e\}$ is part of some MST.

- cut property guarantees that it is always safe to add the lightest edge across any cut, provided it is compatible with X.

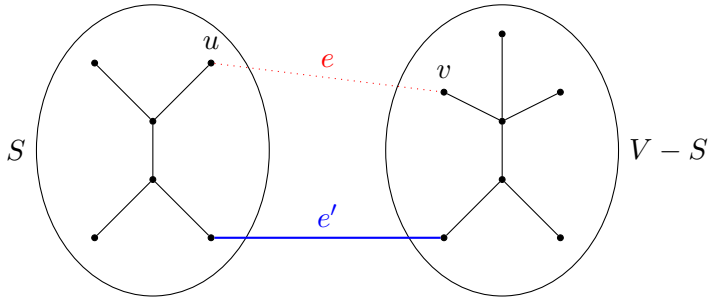
Proof of Cut Property (1/2)

Suppose the set of edges X is part of some MST T (partial solution, on the right track).

- If the new edge e also happens to be part of T, then there is nothing to prove.
- If $e \notin T$, we can construct a different MST T' containing $X \cup \{e\}$ by modifying T slightly, changing just one of its edges.
  1. Add $e = (u, v)$ to T. T is connected, it already has a path between u and v, adding e creates a cycle.
     - This cycle must also have some other edge e' across the cut $(S, V - S)$.
  2. Remove edge e', we are left with $T' = T \cup \{e\} - \{e'\}$:

Proposition 1 + Proposition 2 $\Rightarrow$ T' is still a spanning tree

Proof of Cut Property (2/2)



It remains to prove T' is also an MST.

Proof idea. Compare its weight to that of T

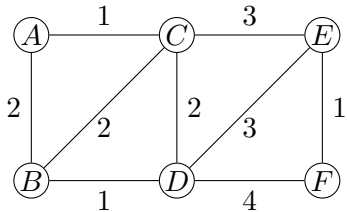
$$\text{weight}(T') = \text{weight}(T) + w(e) - w(e')$$

Both e and e' cross between S and $V - S$, and e is the lightest edge of this type.

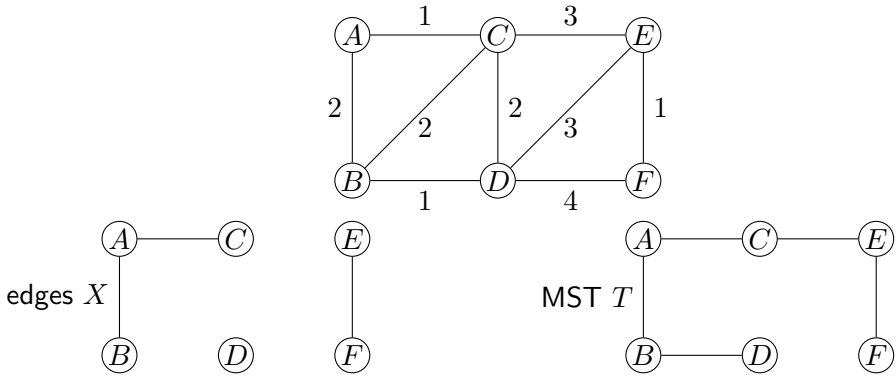
$$w(e) \leq w(e') \Rightarrow \text{weight}(T') \leq \text{weight}(T)$$

$$T \text{ is an MST} \Rightarrow \text{weight}(T) = \text{weight}(T') \Rightarrow T' \text{ is also an MST}$$

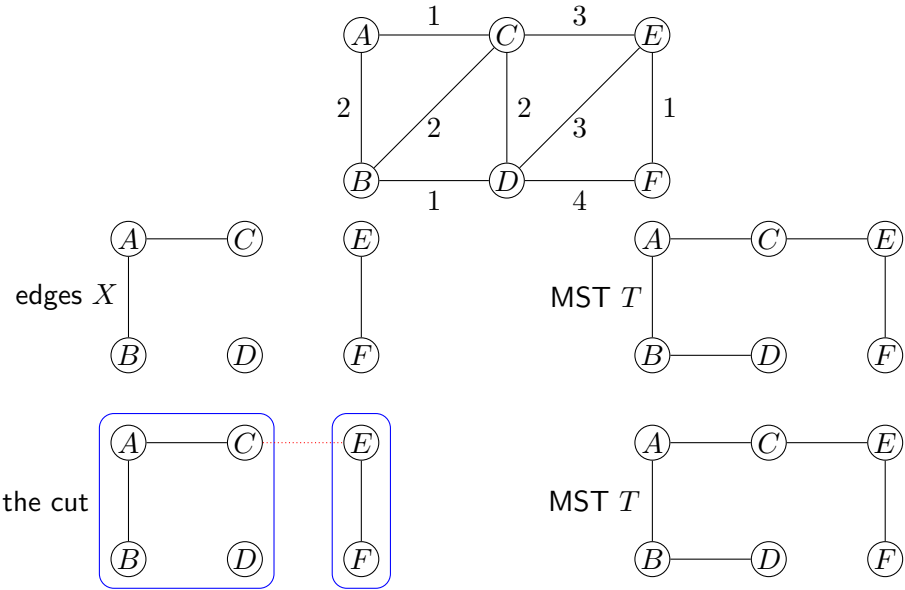
The Cut Property at Work



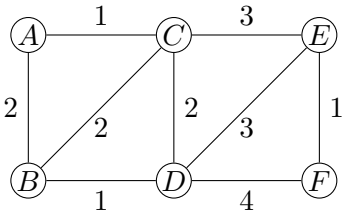
The Cut Property at Work



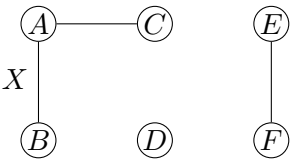
The Cut Property at Work



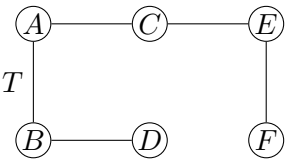
The Cut Property at Work



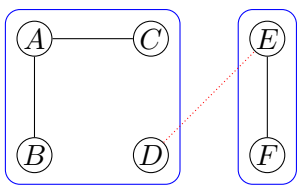
edges X



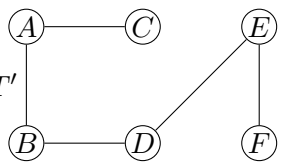
MST T



the cut



MST T'



General MST Algorithm Based on Cut Property

Algorithm 3: GeneralMST($G = (V, E)$): output MST defined by X

1: $X = \emptyset$;
2: **while** $|X| < |V| - 1$ **do**
3: pick a set $S \subset V$ such that X has no edge crossing $(S, V - S)$; `// find a compatible cut`
4: let $e \in E$ be the minimal-weight edge that crosses S and $V - S$;
5: $X \leftarrow X \cup \{e\}$;
6: **end**

Next, we describe two famous MST algorithms following this template.

Outline

1. Huffman Coding
2. Single Source Shortest Path Problem
   - Dijkstra's Algorithm
3. Minimum Spanning Tree
   - The Cut-Based MST Algorithm
   - Kruskal's Algorithm
   - Prim's Algorithm

Kruskal's Algorithm (edge-by-edge)

Joseph Kruskal [American Mathematical Society, 1956]

Kruskal's Algorithm (edge-by-edge)

Joseph Kruskal [American Mathematical Society, 1956]

Rough idea. Start with the empty graph, then select edges from E according to the following rule

Repeatedly add the next lightest edge that doesn't produce a cycle

Kruskal's Algorithm (edge-by-edge)

Joseph Kruskal [American Mathematical Society, 1956]

Rough idea. Start with the empty graph, then select edges from E according to the following rule

Repeatedly add the next lightest edge that doesn't produce a cycle

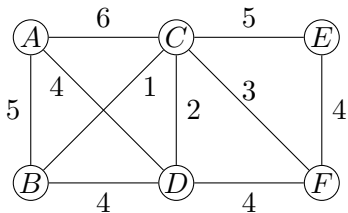
---

Kruskal's algorithm constructs the tree *edge-by-edge*, apart from taking care to avoid cycles, simply picks the lightest edge at the moment.

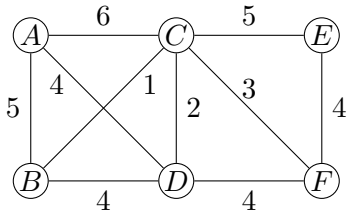
- This is a greedy algorithm: every decision it makes is the one with the most obvious immediate advantage.

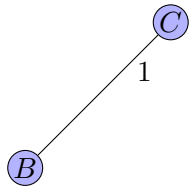
X is initially empty, check every possible cut $(S, V - S)$.

Demo of Kruskal's Algorithm

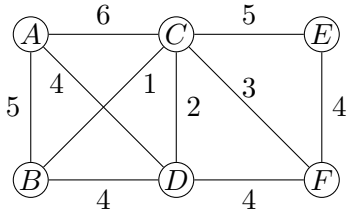


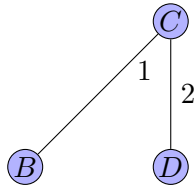
Demo of Kruskal's Algorithm



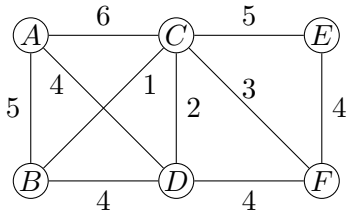


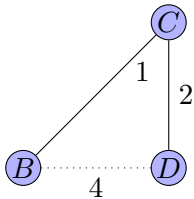
Demo of Kruskal's Algorithm



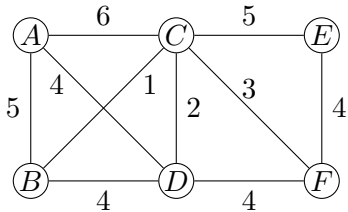


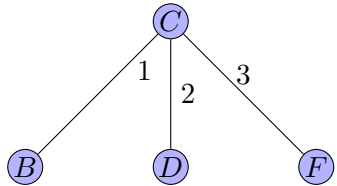
Demo of Kruskal's Algorithm



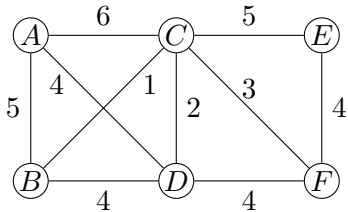


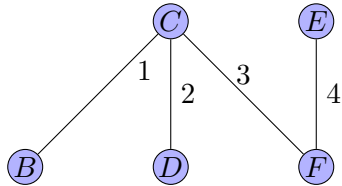
Demo of Kruskal's Algorithm



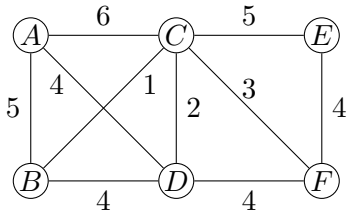


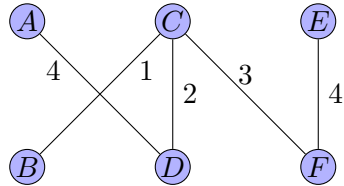
Demo of Kruskal's Algorithm





Demo of Kruskal's Algorithm





Interpretation of Kruskal's Algorithm from Cut Perspective

At the very beginning, view $X = \emptyset$ as n disjoint components.

At any given moment, the set of edges X it has already chosen corresponds to:

- a partial solution T': a collection of connected components, each of which has a tree structure

The next edge e to be added connects two of these components, say, T_1 and T_2

- viewing T_1 as $S \Rightarrow (T_1, V - T_1)$ forms a cut compatible with X
  - before adding e, T_1 and $V - T_1$ were not connected, thus no edge in X cross T_1 and $V - T_1$
- According to the design, e is the lightest edge that does not produce a cycle, it is certainly to be the lightest edge between T_1 and $V - T_1$

Kruskal's algorithm implicitly searches the lightest crossed edge among all possible compatible cuts

Implementation Details

Select-and-Check. At each stage, the algorithm chooses an edge to add to its current partial solution.

- To do so, it needs to test each candidate edge (u, v) to see whether u and v lie in different components $\Rightarrow$ otherwise the edge produces a cycle

This test implicitly ensures that X is compatible with any cut combination of the form

$$(T_i, V - T_i)$$

Merge. Once an edge is chosen, the corresponding components need to be merged.

Which data structure supports such operations?

Data Structure

Disjoint-Set Union-Find. Efficiently manages a collection of disjoint sets. Its primary function is to track which elements are connected and to merge sets together.

- find(x): determine which set an element x belongs to, return a "representative" element of that set.
- union(x, y): merge the set containing element x with the set containing element y.

Efficiency: With optimizations, both operations can run in nearly constant time, specifically $O(\alpha(n))$, where α is the inverse Ackermann function, a function that grows extremely slowly.

Simplicity: It is very easy to code, often requiring less than 20 lines.

Union-Find for Kruskal's Algorithm

Model algorithm's state as a collection of *disjoint sets*, each containing the nodes of a particular component.

Initially each node is a component by itself.

- makeset(x): create a singleton set containing just x

Repeatedly test pair of nodes (endpoints of candidate edge) to see if they belong to the same set.

- find(x): to which set does x belong?

Whenever we add an edge, merge two components

- union(x, y): merge the sets containing x and y

Pseudocode of Kruskal's Algorithm

Algorithm 4: Kruskal(G): output MST defined by X

```
1: sort the edges E by weight, X = ∅;
2: for all u ∈ V do makeset(u);
3: while |X| < |V| − 1 do
4:     for all edges (u, v) ∈ E (in ascending order) do
5:         E ← E \ {(u, v)} ;        // always remove the edge under check
6:         if find(u) ≠ find(v) then // check if X is compatible with cut
7:             X ← X ∪ {(u, v)}, union(u, v), break;
8:         end
9:     end
10: end
```

Complexity Analysis

- cost of sorting E: $O(|E| \log |E|)$; cost of makeset(u): $O(|V|)$
- total cost of find(x): $2|E| \cdot O(\text{find})$ (independent of `while`)
- union: $(|V| - 1) \cdot O(\text{union})$

Outline

1. Huffman Coding
2. Single Source Shortest Path Problem
   - Dijkstra's Algorithm
3. Minimum Spanning Tree
   - The Cut-Based MST Algorithm
   - Kruskal's Algorithm
   - Prim's Algorithm

Prim's Algorithm (node-by-node)

First discovered by Czech mathematician Vojtěch Jarník 1930, later rediscovered and republished by Robert C. Prim in 1957, and Edsger W. Dijkstra in 1959. Thus, known as the DJP algorithm.

Rough idea

1. Initially: $X = \emptyset$, $S = \{u_0\}$, u_0 could be an arbitrary node
2. Greedy choice: At each step, select the lightest edge $e_{u,v}$ that connects S and $V - S$, where $u \in S$, $v \in V - S$. Add $e_{u,v}$ to X, add v to S.
3. Continue the procedure until $S = V$.

Prim's algorithm is an alternative to Kruskal's algorithm, another implementation of the General cut-based algorithm

- X is initially empty, S is initially any node
- the graph (S, X) always forms a subtree, S is the vertices set of X after first step. This choice makes $(S, V - S)$ naturally constitute a compatible cut w.r.t. X.

Prim's Algorithm (node-by-node)

We can equivalently think of S as growing to include $v^* \notin S$ of smallest cost.

$$\text{cost}(v^*) = \min_{u \in S} w(u, v^*)$$

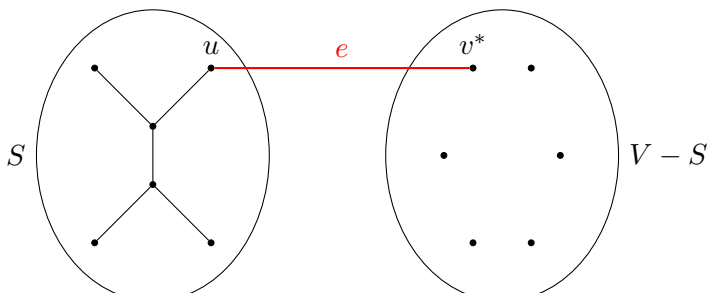
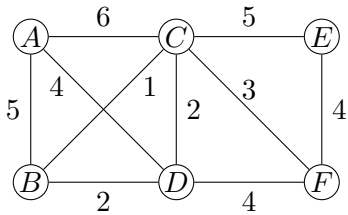


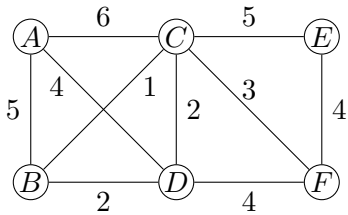
Figure: $T = (S, X)$ forms a tree, and S consists of its vertices.

Demo of Prim's Algorithm



Set S	A	B	C	D	E	F
A		$5/A$	$6/A$	$4/A$	$\infty/\perp$	$\infty/\perp$
A, D		$2/D$	$2/D$		$\infty/\perp$	$4/D$
A, D, B			$1/B$		$\infty/\perp$	$4/D$
A, D, B, C					$5/C$	$3/C$
A, D, B, C, F					$4/F$	

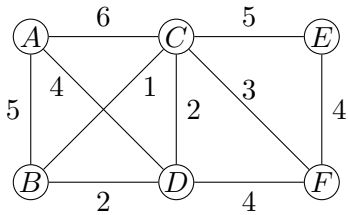
Demo of Prim's Algorithm

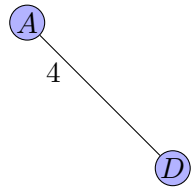




Set S	A	B	C	D	E	F
A		$5/A$	$6/A$	$4/A$	$\infty/\perp$	$\infty/\perp$
A, D		$2/D$	$2/D$		$\infty/\perp$	$4/D$
A, D, B			$1/B$		$\infty/\perp$	$4/D$
A, D, B, C					$5/C$	$3/C$
A, D, B, C, F					$4/F$	

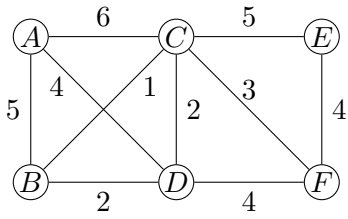
Demo of Prim's Algorithm

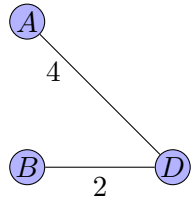




Set S	A	B	C	D	E	F
A		$5/A$	$6/A$	$4/A$	$\infty/\perp$	$\infty/\perp$
A, D		$2/D$	$2/D$		$\infty/\perp$	$4/D$
A, D, B			$1/B$		$\infty/\perp$	$4/D$
A, D, B, C					$5/C$	$3/C$
A, D, B, C, F					$4/F$	

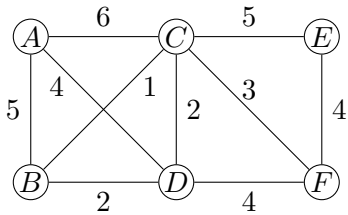
Demo of Prim's Algorithm

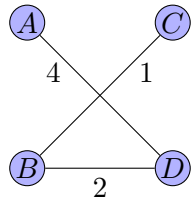




Set S	A	B	C	D	E	F
A		$5/A$	$6/A$	$4/A$	$\infty/\perp$	$\infty/\perp$
A, D		$2/D$	$2/D$		$\infty/\perp$	$4/D$
A, D, B			$1/B$		$\infty/\perp$	$4/D$
A, D, B, C					$5/C$	$3/C$
A, D, B, C, F					$4/F$	

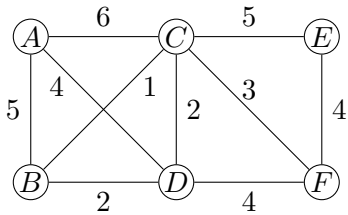
Demo of Prim's Algorithm

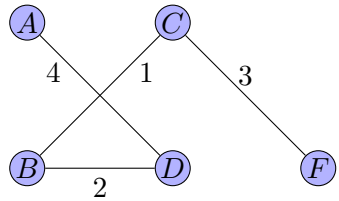




Set S	A	B	C	D	E	F
A		$5/A$	$6/A$	$4/A$	$\infty/\perp$	$\infty/\perp$
A, D		$2/D$	$2/D$		$\infty/\perp$	$4/D$
A, D, B			$1/B$		$\infty/\perp$	$4/D$
A, D, B, C					$5/C$	$3/C$
A, D, B, C, F					$4/F$	

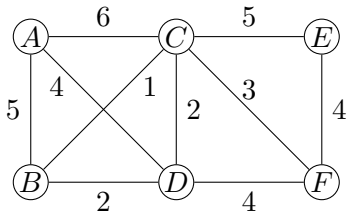
Demo of Prim's Algorithm

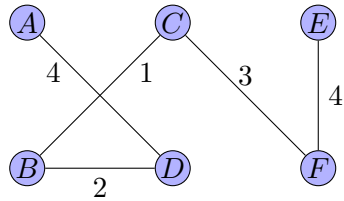




Set S	A	B	C	D	E	F
A		$5/A$	$6/A$	$4/A$	$\infty/\perp$	$\infty/\perp$
A, D		$2/D$	$2/D$		$\infty/\perp$	$4/D$
A, D, B			$1/B$		$\infty/\perp$	$4/D$
A, D, B, C					$5/C$	$3/C$
A, D, B, C, F					$4/F$	

Demo of Prim's Algorithm





Set S	A	B	C	D	E	F
A		$5/A$	$6/A$	$4/A$	$\infty/\perp$	$\infty/\perp$
A, D		$2/D$	$2/D$		$\infty/\perp$	$4/D$
A, D, B			$1/B$		$\infty/\perp$	$4/D$
A, D, B, C					$5/C$	$3/C$
A, D, B, C, F					$4/F$	

Key Data Structure

At every step, Prim's algorithm has to find the lightest edge that connects S and $V - S$.

How to implement this operation? Which data structure supports this operation?

We use priority queue.

Pseudocode of Prim's Algorithm

Algorithm 5: Prim(G): output MST defined by array prev

1: **for** *all* $u \in V$ **do** cost(u) $= \infty$, prev(u) $= \perp$;
2: pick any initial node u_0: cost(u_0) $= 0$;
3: $Q =$ makequeue(V), $S \leftarrow \emptyset$;
4: **while** Q *is not empty* **do**
5: $v^* =$ extractmin(Q) ; `// current closest point in` $V - S$ `to` S
6: $S \leftarrow S \cup \{v^*\}$;
7: **for** *each* $v \in V - S$ **do**
8: **if** $w(v^*, v) <$ cost(v) **then**
9: cost(v) $= w(v^*, v)$, prev(v) $= v^*$; `// update after adding` v^*
10: decreasekey(Q, v)
11: **end**
12: **end**
13: **end**

- Q is a priority queue, using cost-value as keys.

Correctness Proof of Prim's Algorithm: Mathematical Induction

Proposition. For any $k < n$, there exists a MST containing the edges selected by Prim's algorithm in the first k steps.

- Prim's algorithm selects one edge in each step, selects $n - 1$ edges in total. Thus, the proposition proves the correctness of Prim's algorithm.

Proof sketch. Mathematical induction on steps.

Induction basis. $k = 1$, there exists a MST T that contains $e_{u,i}$, where $e_{u,i}$ is the minimal-weight edge connected to node u.

Induction step. Assume the edges selected by the first k steps form a subset of some MST, so does the first $k + 1$ steps.

Induction Basis

Claim: There exists a MST T that contains the minimal-weight edge $e_{u,i}$.

Proof. Let T be a MST. If T does not contain $e_{u,i}$, then $T \cup \{e_{u,i}\}$ must contain a cycle, and the cycle has another edge $e_{u,j}$ connecting to node u. Replacing $e_{u,j}$ with $e_{u,i}$ we obtain T^*, T^* is also a spanning tree.

- If $e_{u,i} < e_{u,j}$, then weight$(T^*) <$ weight(T). This contradicts to the hypothesis that T is a MST.
- If $e_{u,i} = e_{u,j}$, then weight$(T^*) =$ weight(T). Then, T^* is a MST that contains $e_{u,i}$.

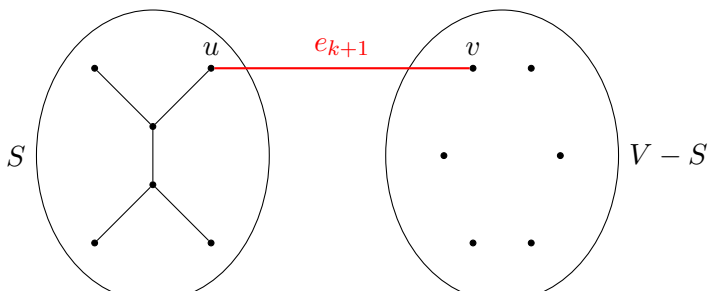


Induction Steps (1/2)

After k steps, Prim's algorithm selects edges $e_1, e_2, \ldots, e_k$. The nodes of these edges form a node set S.

Premise. $\exists$ MST $T = (V, E)$ that contains $(e_1, \ldots, e_k)$.

Let the $k+1$ step choice be $e_{k+1} = (u, v)$, $u \in S$, $v \in V - S$.

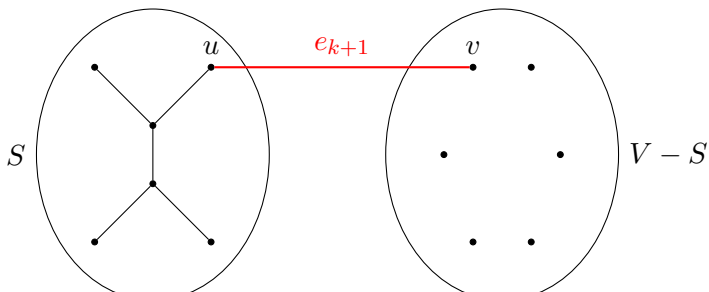


Induction Steps (1/2)

After k steps, Prim's algorithm selects edges $e_1, e_2, \ldots, e_k$. The nodes of these edges form a node set S.

Premise. $\exists$ MST $T = (V, E)$ that contains $(e_1, \ldots, e_k)$.

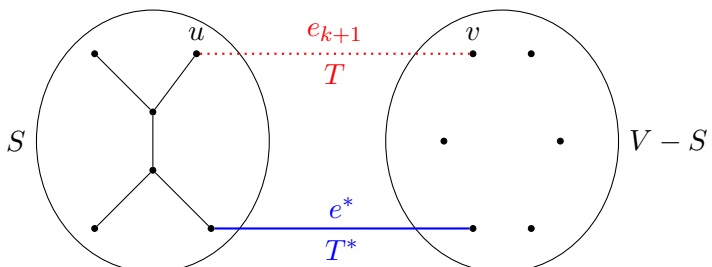
Let the $k+1$ step choice be $e_{k+1} = (u, v)$, $u \in S$, $v \in V - S$.



Case $e_{k+1} \in E$: the induction step of Prim's algorithm at $k+1$ step is obviously correct.

Induction Step (2/2)

Case $e_{k+1} \notin E$: adding e_{k+1} to E would create a cycle between (u, v). In this cycle, $\exists$ another edge e^* connecting S and $V - S$.

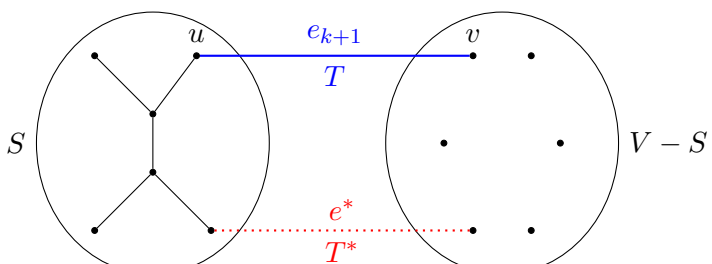


Induction Step (2/2)

Case $e_{k+1} \notin E$: adding e_{k+1} to E would create a cycle between (u, v). In this cycle, $\exists$ another edge e^* connecting S and $V - S$.

Let $T^* = (V, E \setminus \{e^*\}) \cup \{e_{k+1}\})$, then T^* is also a spanning tree of G, which consists of $e_1, \ldots, e_k, e_{k+1}$.

- If $e_{k+1} < e^*$, then weight$(T^*) <$ weight(T). This contradicts to the hypothesis that T is a MST.
- If $e_{k+1} = e^*$, then weight$(T^*) =$ weight(T). T^* is also a MST, the output after $k + 1$ steps is still a subset of T^*.



Kruskal's Algorithm vs. Prim's Algorithm

Kruskal's algorithm grows a forest component by component

- initial state: $X = \emptyset$, $V(\text{MST}) = \emptyset$
- growth of MST: X always forms a subgraph of one predetermined final MST
- cut property: try all possible cuts compatible with X
- data structure: union set

Prim's algorithm grows a single tree from an initial vertex

- initial state: $X = \emptyset$, $V(\text{MST}) = \forall u$
- growth of MST: X always forms a subtree of one predetermined final MST
- cut property: select a particular cut determined by X (S is initially an arbitrary vertice, then the vertex set of X)
- data structure: priority queue

Summary of This Lecture

Greedy strategy: Applicable to many optimization problems (simple and efficient)

- build up a solution piece by piece
- always choose the next piece that offers the most obvious and immediate benefit (rely on heuristic)

How to (dis)prove correctness of greedy algorithm? (counter-example)

- Mathematical induction (on algorithm steps or input size)
- Exchange argument (gradually transform optimal solution to algorithm solution without affecting optimality)

Some classical greedy algorithms: Huffman coding, Dijkstra's algorithm, Prim's algorithm, Kruskal's algorithm